

Informatics II, Spring 2026, Exercise 9

Publication of exercise: May 4, 2026

Publication of solution: May 11, 2026

Exercise classes: May 11 - 15, 2026

Task 1 (Easy)

Consider a hash table with 11 slots, and use chaining to resolve conflicts. For the hash function $h_1(k)$ below, draw the hash table after inserting the following values in the given order: 6, 30, 41, 25, 17, 10, 60, 14, 52, 5. *Note: Insert newer elements at the end of the linked list.*

(a) $h_1(k) = (k + 5) \bmod 11$

Task 2 (Easy)

Now consider a Hash Table with open addressing. Insert the values {6, 30, 41, 25, 17, 10, 60, 15, 52, 5} using the provided hash functions $h_3(k, i)$ and $h_4(k, i)$.

(a) $h_3(k, i) = (k + i) \bmod 11$

(b) $h_4(k, i) = (k \bmod 11 + i * (k \bmod 7)) \bmod 11$

Task 3: Implement hash tables in C (Medium-Hard)

Implement a hash table with m slots, where m is a positive integer. All keys are strict positive integers. Use open addressing with linear probing with step size of 1 to resolve collisions:

$$h(k, i) = (3k + i) \bmod m$$

The following hints will help you through the implementation.

- (a) Define the number m that defines the size of the hash table. Set m to 7.
- (b) The function `void init(int A[])` fills all slots of the hash table A with the value 0 (default value for an empty slot).
- (c) The hashing function `int h(int k, int i)` that receives the key k and the probe number i and returns the hashed key.
- (d) The function `void insert(int A[], int key)` inserts the key into the hash table A.

- (e) The function `int search(int A[], int key)` that returns `-1` if the key was not found in the hash table `A`. Otherwise, it should return the index of the key in the hash table.
- (f) The function `void printHash(int A[])` prints the table size and all slots of the hash table `A` accompanied with their index and the key.
- For testing purposes, use a table size of 7, add the values 4, 9, 11, 16 & 5 and print your hash table. Search for the values 5 & 10 and print the results.
- (g) The C code function `int HTDelete(int k)`, should delete key k from the hash table and also rearrange the other elements. The rearrangement leaves the table exactly as it would have been had key k never been inserted i.e., you cannot simply set the value to 0.
- (a) What will the following hash table look like after deleting key 5. The hash function h given below was used for insertion.

Slot	0	1	2	3	4	5	6	7	8	9	10
Value	0	0	0	1	5	12	9	6	13	0	0

 $h(k, i) = (3k + i) \bmod 11$

- (b) Implement the function `int HTDelete(int k)`. Make sure to leverage the functions you have created in the previous steps. For testing purposes delete 9 & 11 after the insertions from above.

Task 4 (Medium)

Describe how hash tables could be used to solve the following problems. Provide a small explanation and argumentation, followed by a pseudocode implementation. You may assume that basic functions to perform hash table operations have been implemented for you (insert, search, delete), similar to your implementations in Task 3.

- (a) **Duplicates**
You are given two arrays A and B with respective lengths n and m containing strictly positive integers. Write an algorithm that prints to the console all elements that appear in both arrays ($A \cap B$). The algorithm should run in $O(n + m)$.
- (b) **Target Sum** You are given an array A with n strictly positive, distinct integers and a target sum t . Write an algorithm to find if any two integers in A form the target sum t . The algorithm should run in $O(n)$ time and return 1 if it is possible to form t , and 0 otherwise.